Mitchell McNaney

CS 470 Fullstack 2

Final Reflection – 10/18/2026

https://youtu.be/gXZwFyprxrw

This course was an excellent way to round out my skills as a developer. I felt like it helped me understand deployment as a whole much better. I've focused a lot on the actual code, performance, and security up until now, and this class helped me learn about containerization, cloud-computing, serverless architecture, docker/docker compose, and how to deploy full-stack applications more effectively. This gave me practical experience working with AWS Lambda, API Gateway, DynamoDB, and other cloud services.

As I prepare to enter the software development field, these skills will make me a more competitive candidate. I now have experience with both traditional full-stack development and cloud-based application architectures. One of my strengths as a developer is my ability to troubleshoot problems and work through unfamiliar technologies. Throughout my coursework, I have worked with several programming languages, frameworks, and development environments, which has helped me become more adaptable. I also enjoy identifying opportunities to improve efficiency through automation and better software design.

With these experiences, I feel prepared to pursue entry-level positions in software development, particularly roles involving backend development, full-stack development, or cloud-based applications. I know that I still have a lot to learn, but this course has given me a foundation for working with technologies commonly used in professional development environments.

One of the biggest advantages of cloud computing is the ability to scale applications as demand increases. For my application, I would continue using AWS Lambda and API Gateway because they allow backend services to scale automatically without managing servers. Breaking functionality into smaller services also makes updates and troubleshooting easier. I would use CloudWatch to monitor errors and performance, along with exception handling and retry mechanisms to improve reliability. Cost would also be an important consideration. AWS Pricing Calculator and Cost Explorer could help estimate and track expenses. Containers generally provide more predictable costs for consistent workloads, while serverless can be more affordable for applications with unpredictable traffic because you primarily pay for what you use.

Serverless offers easier management and automatic scaling, but limitations like cold starts and execution timeouts could become concerns. Containers offer more control but require additional maintenance. For now, I would favor serverless because its elasticity and pay-for-service model allow the application to grow without requiring significant upfront infrastructure costs. As usage increases, I would reevaluate which approach offers the best balance of performance and cost.

References:

Mao, G. (2019, June 25). How to design your serverless apps for massive scale. AWS Architecture Blog. https://aws.amazon.com/blogs/architecture/how-to-design-your-serverless-apps-for-massive-scale/

Cloudflare. (n.d.). Serverless computing vs. containers: How to choose. https://www.cloudflare.com/learning/serverless/serverless-vs-containers/